AI Anticipates the Asking Price

City University of Seattle
School of Technology & Computing
TEAM A

Verónica Elze
ElzeVeronica@CityUniversity.edu
Master of Artificial Intelligence

Apoorva Guru
GuruApoorva@CityUniversity.edu
Master of Data Science

Teagan Scharlau
TScharlau@CityU.edu
Doctor of Information Technology

George Stanley
YanHao@CityUniversity.edu
Doctor of Information Technology

Abstract

This project will explore the development of an environment to host an AI application on a cloud platform. The core objective is to run multiple artificial intelligence (AI) algorithms, each deployed as a container and utilize either Flask or FastAPI frameworks to serve as prediction services. The AI capabilities integrated into our application will focus primarily on market analysis and possibly price prediction and automated property valuation.

In market analysis, the AI algorithms will analyze vast datasets to identify trends and patterns, offering insights that can guide strategic decision-making. Price prediction algorithms would use historical data and various market indicators to forecast future prices with a high degree of accuracy. This feature would be particularly useful for investors and individual buyers seeking to anticipate market movements. Automated property valuation would leverage AI to assess property values based on multiple factors, including location, market conditions, and property features. This functionality aims to provide accurate and reliable property valuations, helping both buyers and sellers in making informed decisions. By integrating these AI capabilities, our application enhances market understanding and empowers users with predictive insights and data-driven forecasts. By using cloud infrastructure, we ensure that our AI models can scale seamlessly to handle varying loads, providing consistent performance regardless of user demand and ensure that the application remains accessible.

Keywords: artificial intelligence, machine learning, cloud service, predictive analytics, market predictions, real estate

1. INTRODUCTION

In modern real estate, both future homeowners and investors face numerous challenges. Key questions such as whether property prices align with comparable properties, whether prices are trending up or down, and what the future holds based on historical trends are difficult to answer due to market fluctuations. These uncertainties can cause investors and home buyers to miss optimal times for making real estate decisions. To address these time-sensitive issues and empower property buyers, artificial intelligence (AI) offers promising solutions.

AI can significantly enhance the ability to identify investment opportunities and estimate property costs in specific locations with particular features. Its predictive modeling capabilities allow for the assessment and forecasting of future trends and conditions, aligning perfectly with the needs of our research study. Machine Learning (ML) models, accessible via application programming interfaces (APIs), can receive user information, apply the model, and predict housing prices in specific markets. This reduces uncertainty around price comparisons and trends in target market areas, predicting whether prices are ascending or descending. Depending on the model's training, AI can also identify more or less desirable markets for investors and home buyers.

Zillow, for example, has been utilizing AI since 2006 to benefit both customers and agents (Our AI Principles - Zillow Group, 2024). For consumers, AI provides instant home estimates, assists with home finance estimation, and compares comparable properties in the area to ensure pricing is in line with the market. For agents, AI helps scale operations as the market grows, automates mundane and repetitive tasks, and provides access to important data about properties and market trends.

This literature review will delve into the background of AI-driven pricing, compare various real estate prediction studies, and examine the cloud computing systems that support these technologies. By understanding the current landscape and potential hazards associated with AI-driven pricing, stakeholders can make more informed decisions and leverage AI to its fullest potential in the real estate market.

2. LITERATURE REVIEW

The integration of artificial intelligence (AI) in the real estate market has revolutionized how properties are valued and priced. AI-driven pricing leverages sophisticated algorithms to analyze historical transaction data, property features, and market dynamics, generating precise valuations that enhance decision-making processes for real estate professionals (Clement, 2024). This literature review explores AI and predictive analytics in real estate transactions, examining studies and comparing cloud computing systems that support these technologies.

Background on AI-driven pricing

The implementation of AI-driven pricing in the real estate industry presents several potential hazards. A comprehensive literature review reveals the state of AI and predictive analytics in real estate transactions. Silaghi et al. (2024) conducted a market analysis using deep neural networks trained on Zillow's Zestimate data and other census bureau information to predict price dynamics. They also used a formal parametric model to explain the recursive ripple effects of errors in Market Real Estate Evaluations (MREE) on global inflation.

The study identified several key hazards associated with AI-driven pricing. One significant hazard is the recursive impact of AI products on society and the potential consequences of using AI outputs in training data. This can create a feedback loop where AI-generated price estimations influence market values, affecting future AI predictions and potentially amplifying errors and market distortions.

Understanding the market dynamics influenced by AI price prediction models employed by MREEs like Zillow is crucial. These models can significantly impact market behavior, leading to unintended consequences such as price volatility and market instability. The study also highlights the game-theoretic effects between MREEs and homeowners, particularly in mitigating estimation errors.

In conclusion, while AI-driven pricing offers significant potential for enhancing real estate market efficiency, it also presents several hazards that must be carefully managed. Understanding and addressing these hazards is crucial for leveraging the benefits of AI while minimizing its risks.

Studies about real estate predictions

Predicting rental prices using machine learning and data science has become a significant research area. Two notable studies in this field

are "Housing Market Intelligence: Data Science for Rental Price Forecasting" by JS et al. (2024) and "Machine Learning for Rental Price Prediction: Regression Techniques and Random Forest Model" by Raju et al. (2023). Both aimed to enhance rental price prediction accuracy through advanced machine learning models but differed in their approaches and contexts.

JS et al. (2024) applied a Random Forest Regressor algorithm using property attributes for rental price forecasting, discounting other algorithms like CNN, SVM, Logistic Regression, and KNN due to their unsuitability for the specific data characteristics. They collected diverse datasets from real estate platforms, preprocessed the data, and built a model with multiple decision trees, evaluating predictive accuracy using metrics like MAE, MSE, and R2 score.

Raju et al. (2023) focused on machine learning techniques, regression analysis, and a Random Forest model to predict house rental prices. They used Multiple Linear Regression to model relationships between variables, Ridge Regression to prevent overfitting, and LASSO Regression for feature selection and model simplification. Additionally, they leveraged other regression models like Elastic Net, Gradient Boosting, and Ada Boost Regression for regularization and minimizing errors.

Both studies emphasize the importance of accurate, data-driven methods for forecasting rental prices, using historical rental data and various features to make predictions. However, they differ in their specific techniques and publication contexts. Raju et al. (2023) explored a variety of regression techniques, while JS et al. (2024) focused specifically on the Random Forest Regressor. Despite these differences, both contribute valuable insights into the application of machine learning for rental price prediction.

Cloud Computing Comparisons

Amazon Web Services (AWS) and Microsoft Azure are two of the top cloud platforms, crucial for scalability, efficiency, and innovation. Both offer extensive services but have unique strengths.

Mare et al. (2023) highlighted AWS's vast storage options and pay-as-you-go pricing, while Azure provides competitive storage with discounts for Microsoft customers. Singh (2024) discussed AWS Code Pipeline's integration with Azure DevOps using OpenID Connect, automating build, test, and deploy phases, and linking with GitHub for automatic triggers. Biradar and Moolya (2022) examined GitHub Actions integration with AWS for CI/CD pipelines, using AWS CodeBuild and AWS CodeDeploy for automated builds and deployments.

Both platforms offer computing power, storage, and databases for managing applications at scale. AWS provides Amazon EC2 for scalable virtual servers and AWS Lambda for serverless computing, while Azure offers Virtual Machines and Azure Functions. Both support auto-scaling to adjust resources based on demand.

Integration with DevOps tools is seamless, facilitating continuous integration and deployment. AWS and Azure prioritize security and compliance, offering encryption, identity and access management, and compliance certifications. AWS's early market entry and extensive service array give it a larger global infrastructure footprint and a mature ecosystem. Azure's strong integration with Microsoft's ecosystem appeals to enterprises using Microsoft products.

AWS offers managed services like AWS CodeBuild and AWS CodeDeploy, automating build and deployment processes and integrating with GitHub Actions. Azure provides similar services with Azure DevOps, supporting CI/CD pipelines and integrating with GitHub. AWS's offerings include specialized services for machine learning, analytics, and IoT, while Azure offers competitive alternatives like Azure Machine Learning, Azure Synapse Analytics, and Azure IoT Hub.

Both AWS and Azure provide powerful cloud computing platforms with extensive services, scalability, and robust security. Their differences in service integration, offerings, and user experience highlight their unique strengths. Businesses must consider these factors when choosing the right cloud service provider to meet their needs and objectives.

In summary, this literature review delved into the background of AI-driven pricing, compared various real estate prediction studies, and examined the cloud computing systems that underpin these technologies. By understanding the current landscape and potential hazards associated with AI-driven pricing, stakeholders can make informed decisions and leverage AI to its fullest potential in the real estate market.

3. METHODOLOGY

Methods Applied

For this research project our team conducted a literature review, and examined the current literature around AI, Real Estate, and cost predictions. Next, we developed an AI model and interface for input to and output from the model. We then ran tests on the model for accuracy, and prediction time.

Finally, we analyzed all the data and formalized our findings of the results, which are presented in this report.

Tools & Technologies
For this project we chose to use an existing data set from Kaggle for training and testing the model. It is geared towards housing market research, data analytics and predictive modeling for property prices. (Sukhmandeep Singh Brar, 2024).

To develop our AI model, we used the Python programming language with various libraries, such as TensorFlow, Scikit-learn, Flask, NumPy, and Pandas. Most of the coding work was done either in Visual Studio Code (VS Code) or Google's Collaboratory (Colab) AI Web Platform.

We selected a regression-based algorithm for the model, which can be used to predict continuous numerical values based on user submitting inputs.

The API and User Interface used a combination of Flask, HTM, and JSON. This allows them to be loaded and function within a standard, modern web browser, such as Mozilla Firefox, Google Crome, and Microsoft Edge.

Our Web Interface and back-end will be provided by the Flask module in Python and will use both curl and JSON for data submission and response. This front and back-end will allow for fast development speed, low computational overhead.

We used various standard computer tools to take screenshots, with some figures generated by the Python code that was developed. Additionally, for this report we used Microsoft Word, and for inter-team communication we used Microsoft Teams.

4. MODEL TRAINING PROCESS

Our process for training our ML Model involved multiple steps, covering both data validation and cleaning, model coding and training, model testing, and finally running actual predictions from user input.

Please note, at the time of TP02 this section is preliminary and will be updated as we refine our model, back-end API and deployment, and user interface. The included figures (as well as others referenced) are in Appendix A – Figures and Images at the end of the document in larger format.

Step 1 – Exploration & Data Analysis
The purpose of this step was to examine the dataset, understand its structure, distribution and locate any potential issues.

In machine learning and data analytics, this step is used to identify any patterns in the data, locate outliers in the data points, detect any data quality issues during this preprocessing step.

The initial data load and view of the first five records of the dataset plus the functions executed to examine the data can be found in the appendix as Figures 1, 2, and 3.

We then looked at the distribution of the data set using our target variable of Price:

```
# Visualize the distribution of the target variable (price)
sns.histplot(df['price'], bins=50, kde=True)
plt.title('Distribution of Housing Prices')
plt.show()
```

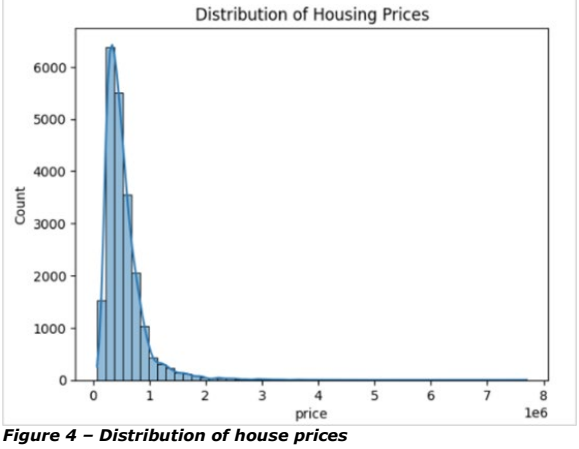


Figure 4 – Distribution of house prices

The final metric we examined about the dataset was the correlation of variables within the dataset. This matrix can be found in the appendix as Figure 5 Visualization of the Correlation Matrix.

Step 2 – Data Preparation
During this step the team cleaned and performed transformations on the data to prepare it for data modeling. This ensured that the dataset is suitable for our ML Algorithm, addresses missing values and encodes categorical variables, while normalizing numerical features. During this step we used the correlation matrix to help reduce features in the dataset, as shown in Figure 6.

```
# Example using correlation threshold
corr_matrix = df.corr()
high_corr_features = corr_matrix.index[abs(corr_matrix['price']) > 0.5]
df_reduced = df[high_corr_features]

df_reduced.head()
```

	price	bathrooms	sqft_living	grade	sqft_above	sqft_living15
0	231300.0	1.00	1180	7	1180	1340
1	538000.0	2.25	2570	7	2170	1690
2	180000.0	1.00	770	6	770	2720
3	604000.0	3.00	1960	7	1050	1360
4	510000.0	2.00	1680	8	1680	1800

Figure 6 – Feature reduction based on correlation matrix

Step 3 – Model Creation & Training
This step was the core of our project process, in that we created the AI/ML model and trained it on a subset of the data to prepare it for use in the

final product. For this project we created different models to test the effectiveness of the models. These models were a linear regression model, a random forest model, and a KNN model.

```
# Initialize models
linear_model = LinearRegression()
random_forest_model = RandomForestRegressor(n_estimators=100, random_state=42)
knn_model = KNeighborsRegressor(n_neighbors=5)

# Separate features and target variable
X = df_reduced.drop('price', axis=1)
y = df_reduced['price']

# Split the data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Train the models
linear_model.fit(X_train, y_train)
random_forest_model.fit(X_train, y_train)
knn_model.fit(X_train, y_train)
```

Figure 7 - Creating and training the models

As shown, we split off 20% of the dataset to be a testing set to run the model against during post-training testing to determine accuracy. We then concluded this step by training the models.

Step 4 – Testing

Machine learning models are not of any use if they are inaccurate or produce unexpected results, and as such one of the most important steps in model development was testing. Therefore, we conducted testing of the models using our previously generated test set from the overall dataset.

```
# Predictions
linear_preds = linear_model.predict(X_test)
rf_preds = random_forest_model.predict(X_test)
knn_preds = knn_model.predict(X_test)

# Evaluation
linear_rmse = np.sqrt(mean_squared_error(y_test, linear_preds))
linear_r2 = r2_score(y_test, linear_preds)

rf_rmse = np.sqrt(mean_squared_error(y_test, rf_preds))
rf_r2 = r2_score(y_test, rf_preds)

knn_rmse = np.sqrt(mean_squared_error(y_test, knn_preds))
knn_r2 = r2_score(y_test, knn_preds)

print(f'Linear Regression RMSE: {linear_rmse}, R^2: {linear_r2}')
print(f'Random Forest RMSE: {rf_rmse}, R^2: {rf_r2}')
print(f'KNN RMSE: {knn_rmse}, R^2: {knn_r2}')
```

Figure 8 – Predicting the models on test data

This produced the results as shown in Figure 9.

```
Linear Regression RMSE: 261392.1693005368, R^2: 0.5480397400391172
Random Forest RMSE: 262715.474127974, R^2: 0.5434520349336687
KNN RMSE: 273653.16468823556, R^2: 0.5046455672573993
```

Figure 9 – RMSE and R^2 values of each model

The RMSE and R^2 values show model performance. As we can see the R^2 shows that the model is about a 50-55% fit to the data, and that in conjunction with the RMSEs means we can do some more to improve the models.

Step 5 – Compare the performance of the different models

To compare our different models, we plotted the Predicted vs Actual values, and the results per model can be found in the following three figures:

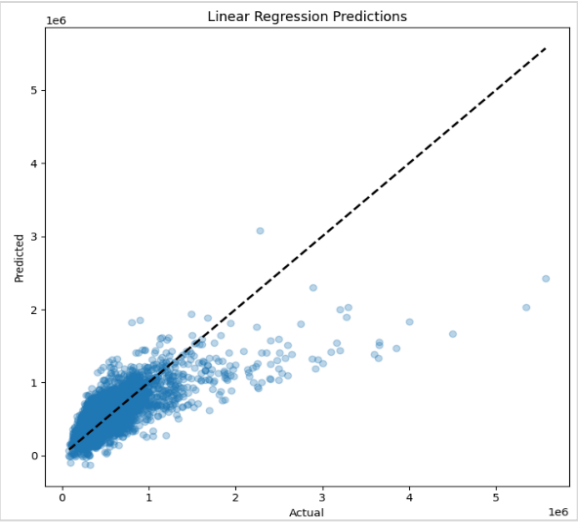


Figure 10 – Prediction Plot: Linear Regression Model

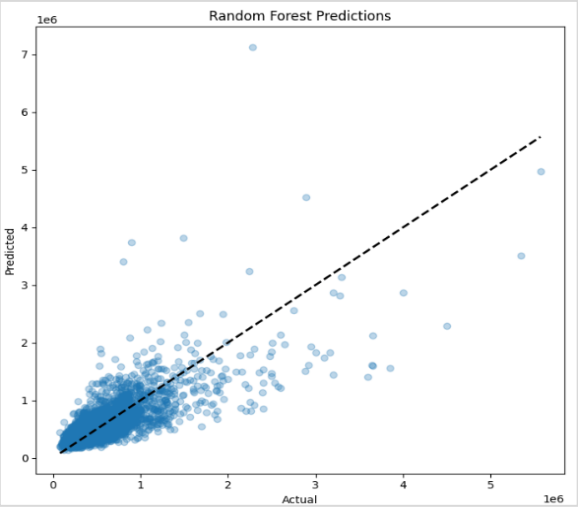


Figure 11 – Prediction Plot: Random Forest

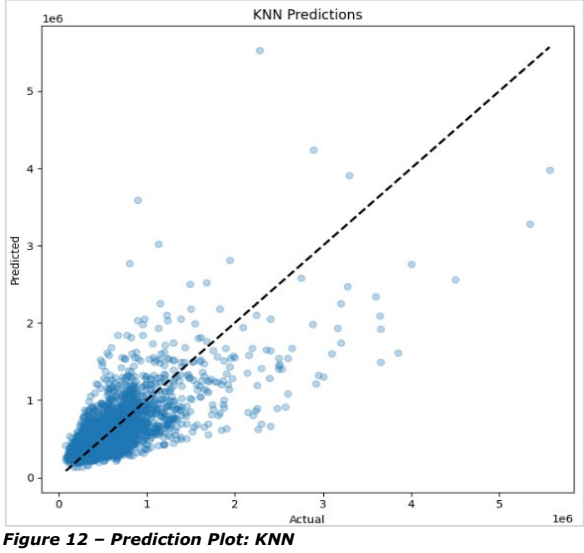


Figure 12 – Prediction Plot: KNN

As we can see from the above plots, the KNN model seems to fit the data better than the other two modes and the data tends to flow along the trend line in the chart, while the other two have points that are well away from the line of best fit.

Microsoft Auto ML:

Microsoft Auto ML (Automated Machine Learning) is a tool designed to automate the end-to-end process of applying machine learning to real-world problems. It simplifies model development by automatically selecting algorithms, tuning hyperparameters, and generating models without requiring deep expertise in machine learning. Integrated with Azure Machine Learning, Auto ML is both accessible and scalable for enterprise use. The team believes in leveraging this cloud platform to achieve the best-performing model.

Exploring Auto ML is essential because it accelerates the model-building process, reduces human error, and allows developers and data scientists to focus on higher-level tasks. This increases productivity and ensures more consistent, accurate results.

Based on performance metrics such as RMSE (Root Mean Square Error) and R^2 (R-Squared), the linear regression model outperformed others. However, this conclusion may change with Auto ML's deeper evaluation and implementation.

The team will pursue the implementation of Auto ML as part of TP03.

4. FINDINGS

Findings will be included in TP03.

5. CONCLUSIONS

Conclusions will be included in TP03.

6. REFERENCES

Al-Said Ahmad, A., & Andras, P. (2019). Scalability analysis comparisons of cloud-based software services. *Journal of Cloud Computing*, *8*(1). https://doi.org/10.1186/s13677-0190134-y

Biradar, M., & Moolya, S. (2022, March 29). *Integrating with GitHub Actions – CI/CD pipeline to deploy a Web App to Amazon EC2*. Amazon Web Services. https://aws.amazon.com/blogs/devops/integrating-with-github-actions-ci-cd-pipeline-to-deploy-a-web-app-to-amazon-ec2/

GitHub Docs. (2024). *Deploying Java to Azure App Service - GitHub Docs*. GitHub Docs. https://docs.github.com/en/actions/deployment/deploying-to-your-cloudprovider/deploying-to-azure/deploying-java-to-azure-app-service

Mansour, F. A. M., Mare, F. A. M., & Abughalyah, A. M. (2023). Amazon AWS vs. Microsoft Azure: Two Prominent Cloud Service Platforms. *International Science and Technology Journal*, *33rd*(1). ResearchGate.https://www.researchgate.net/publication/377774113_Amazon_AWS_vs_Microsoft_Azure_Two_Prominent_Cloud_Service_Platforms

Microsoft. (n.d.). *Tutorial: Train your first machine learning model - Automated ML*. *Microsoft Learn.* https://learn.microsoft.com/en-us/azure/machine-learning/tutorial-first-experiment-automated-ml

Nirmala JS, Somu Geetha Sravya, & Kamsala Tharun. (2024). Housing Market Intelligence: Data Science for Rentalb Price Forecasting. *2024 3rd International Conference for Innovation in Technology (INOCON), 2024*. https://doi.org/10.1109/inocon60754.224.10511646

Raju, R., Arham Neyaz, Ahmed, A., & Singh, A. (2023). Machine Learning for Rental Price Prediction: Regression Techniques and Random Forest Model. *Social Science Research Network*. https://doi.org/10.2139/ssrn.4587725

Silaghi, V., Alssadi, Z., Mathew, B., Alotaibi, M., Alqarni, A., & Silaghi, M. (2024). Modeling the Feedback of AI Price Estimations on Actual Market Values. *ArXiv (Cornell University)*. https://doi.org/10.48550/arxiv.2405.18434

Singh, R. (2024, July 29). *Use OpenID Connect with AWS Toolkit for Azure DevOps to perform AWS CodeDeploy deployments | Amazon Web Services*. Amazon Web Services. https://aws.amazon.com/blogs/devops/use-openid-connect-with-aws-toolkit-for-azure-devops-to-perform-aws-codedeploy-deployments/

Sukhmandeep Singh Brar. (2024). *Housing Price Dataset*. Kaggle.com. https://www.kaggle.com/datasets/sukhmandeepsinghbrar/housing-price-dataset/code

Zillow Group. (2024, January 17). *Our AI principles*. Zillowgroup.com. https://www.zillowgroup.com/our-ai-principles/

APPENDIX A

Tables and Figures – Larger format

```
# Load the dataset
url = "Housing.csv"
df = pd.read_csv(url)
df = df.drop(['id', 'date'], axis=1)
```

```
[54] df.head()
```

	price	bedrooms	bathrooms	sqft_living	sqft_lot	floors	waterfront	view	condition	grade	sqft_above	sqft_basement	yr_built	yr_renovated	zipcode	lat	long	sqft_living15	sqft_lot15
0	231300.0	2	1.00	1180	5650	1.0	0	0	3	7	1180	0	1955	0	98178	47.5112	-122.257	1340	5650
1	538000.0	3	2.25	2570	7242	2.0	0	0	3	7	2170	400	1951	1991	98125	47.7210	-122.319	1690	7639
2	180000.0	2	1.00	770	10000	1.0	0	0	3	6	770	0	1933	0	98028	47.7379	-122.233	2720	8062
3	604000.0	4	3.00	1960	5000	1.0	0	0	5	7	1050	910	1965	0	98136	47.5208	-122.393	1360	5000
4	510000.0	3	2.00	1680	8080	1.0	0	0	3	8	1680	0	1987	0	98074	47.6168	-122.045	1800	7503

Figure 1 - Data Load and Display

```
# Display basic information
print(df.info())
print(df.describe())
```

Figure 2 - Display metadata of loaded data

```
# Check for missing values
print(df.isnull().sum())
```

Figure 3 - Check for nulls of laded dataset

```
# Visualize the distribution of the target variable (price)
sns.histplot(df['price'], bins=50, kde=True)
plt.title('Distribution of Housing Prices')
plt.show()
```

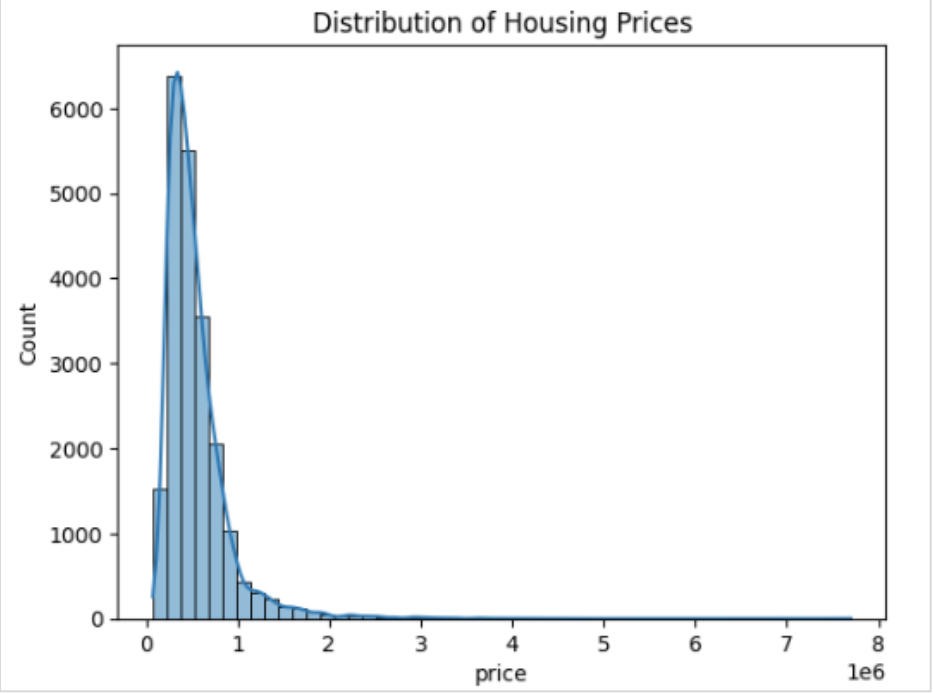


Figure 4 Distribution of house prices

```
# Visualize correlations
plt.figure(figsize=(14, 10))
corr_matrix = df.corr()
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', fmt='.2f')
plt.title('Correlation Matrix', size=20)
plt.show()
```

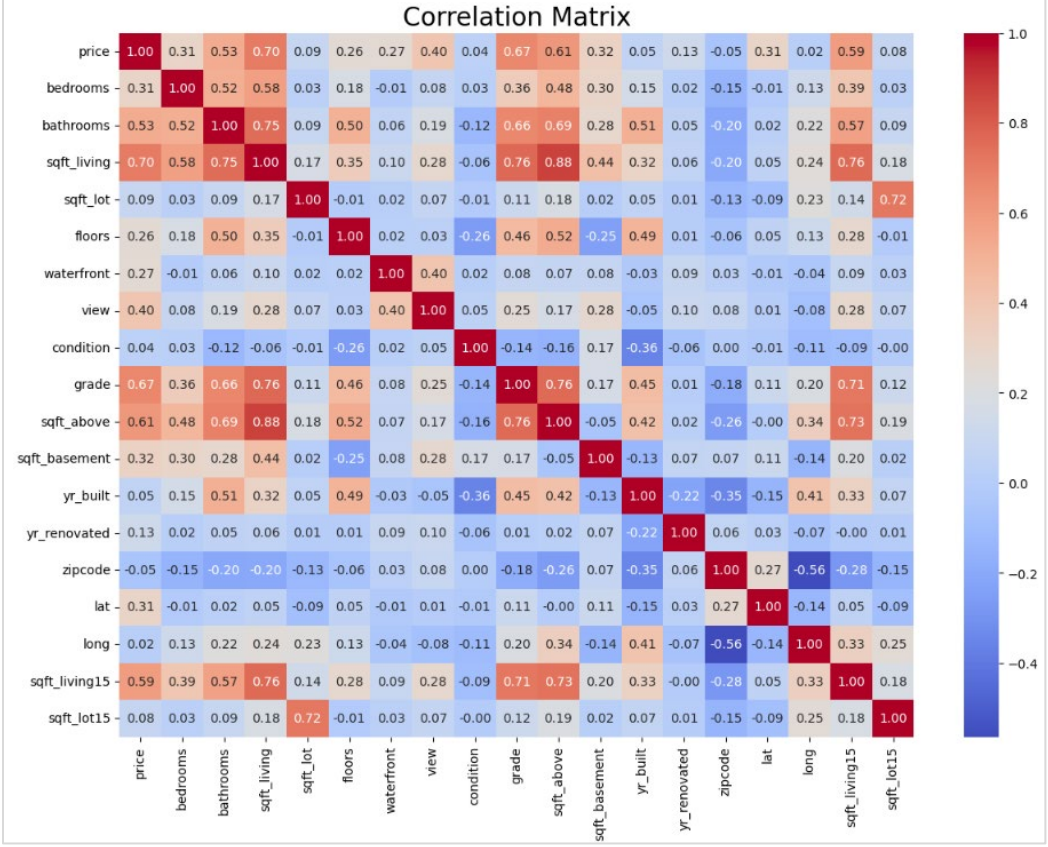


Figure 5 - Visualization of the correlation matrix

```
# Example using correlation threshold
corr_matrix = df.corr()
high_corr_features = corr_matrix.index[abs(corr_matrix['price']) > 0.5]
df_reduced = df[high_corr_features]
```

```
df_reduced.head()
```

	price	bathrooms	sqft_living	grade	sqft_above	sqft_living15
0	231300.0	1.00	1180	7	1180	1340
1	538000.0	2.25	2570	7	2170	1690
2	180000.0	1.00	770	6	770	2720
3	604000.0	3.00	1960	7	1050	1360
4	510000.0	2.00	1680	8	1680	1800

Figure 6 - Feature reduction based on correlation matrix

```python
# Initialize models
linear_model = LinearRegression()
random_forest_model = RandomForestRegressor(n_estimators=100, random_state=42)
knn_model = KNeighborsRegressor(n_neighbors=5)
```

```python
# Separate features and target variable
X = df_reduced.drop('price', axis=1)
y = df_reduced['price']

# Split the data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Train the models
linear_model.fit(X_train, y_train)
random_forest_model.fit(X_train, y_train)
knn_model.fit(X_train, y_train)
```

Figure 7 - Creating and training the models

```python
# Predictions
linear_preds = linear_model.predict(X_test)
rf_preds = random_forest_model.predict(X_test)
knn_preds = knn_model.predict(X_test)

# Evaluation
linear_rmse = np.sqrt(mean_squared_error(y_test, linear_preds))
linear_r2 = r2_score(y_test, linear_preds)

rf_rmse = np.sqrt(mean_squared_error(y_test, rf_preds))
rf_r2 = r2_score(y_test, rf_preds)

knn_rmse = np.sqrt(mean_squared_error(y_test, knn_preds))
knn_r2 = r2_score(y_test, knn_preds)

print(f'Linear Regression RMSE: {linear_rmse}, R^2: {linear_r2}')
print(f'Random Forest RMSE: {rf_rmse}, R^2: {rf_r2}')
print(f'KNN RMSE: {knn_rmse}, R^2: {knn_r2}')
```

Figure 8 - Predicting the models on test data

```
Linear Regression RMSE: 261392.1693005368, R^2: 0.5480397400391172
Random Forest RMSE: 262715.474127974, R^2: 0.5434520349336687
KNN RMSE: 273653.16468823556, R^2: 0.5046455672573993
```

Figure 9 - RMSE, R2 values of the models

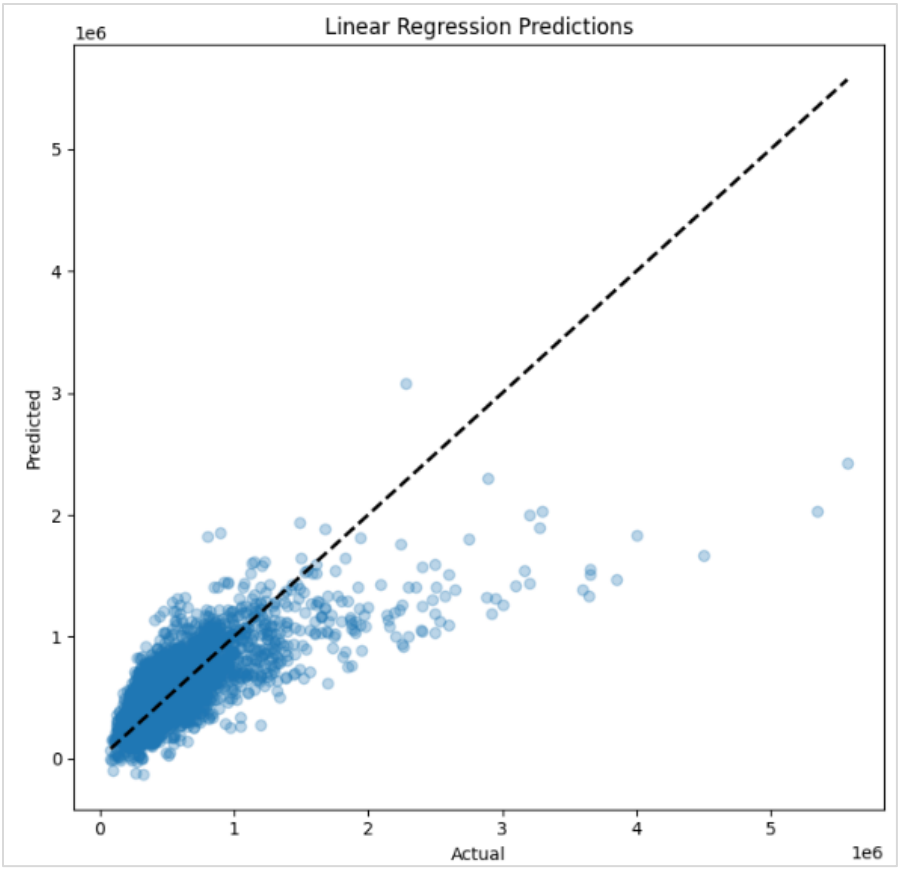


Figure 10 - Prediction Plot: Linear Regression

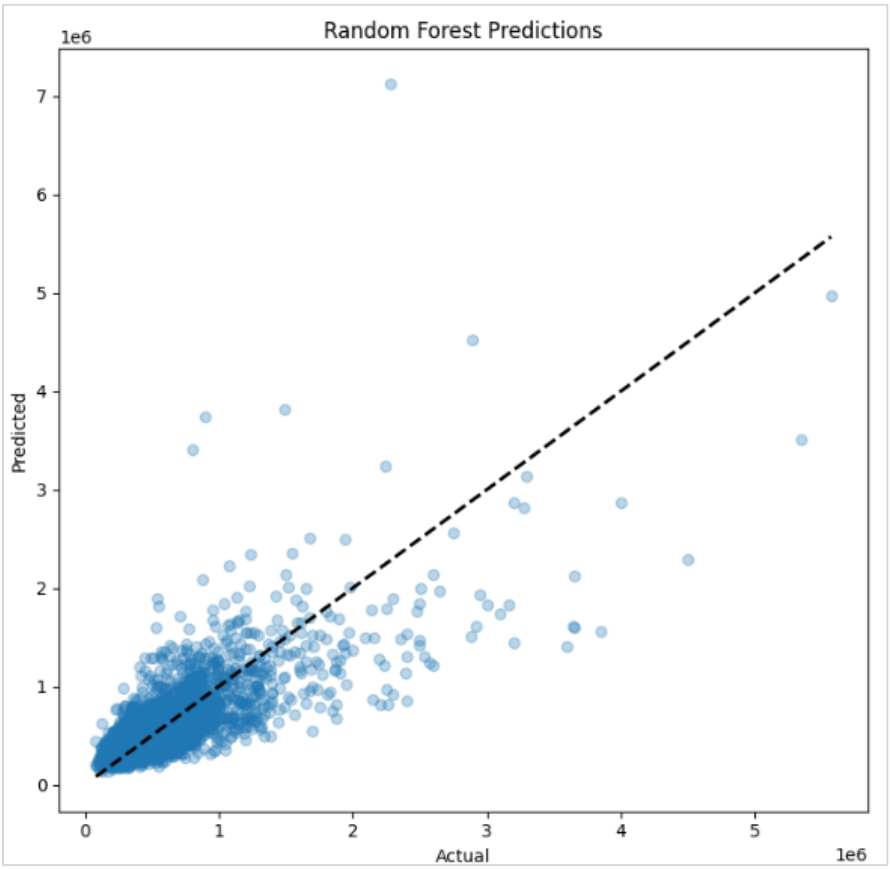


Figure 11 - Prediction Plot: Random Forests

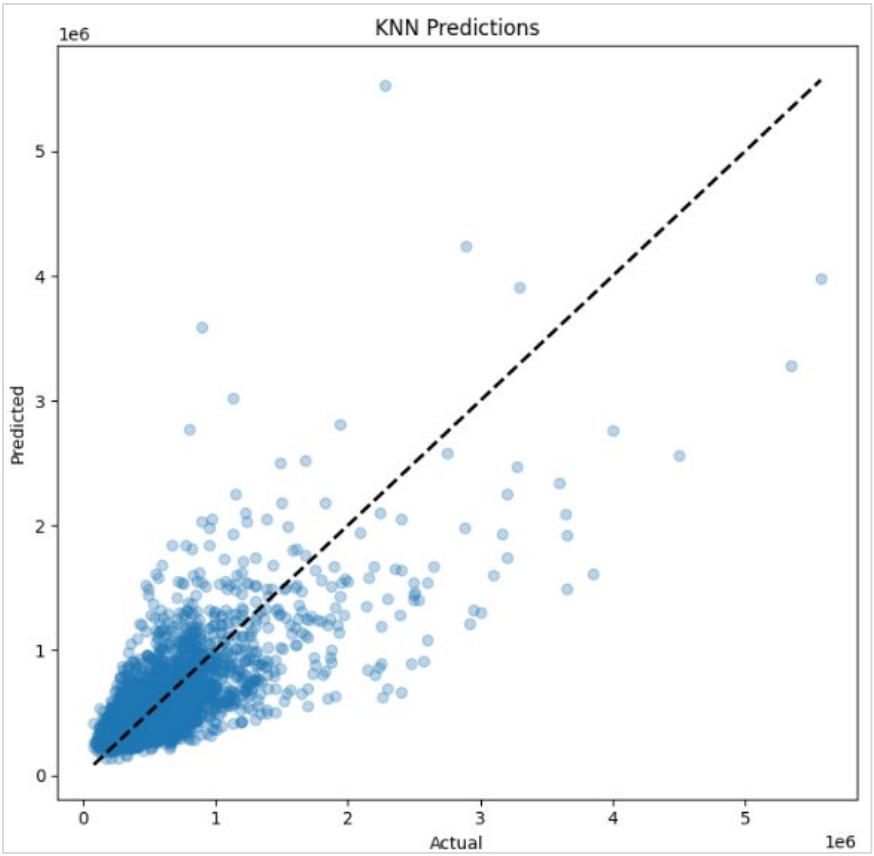
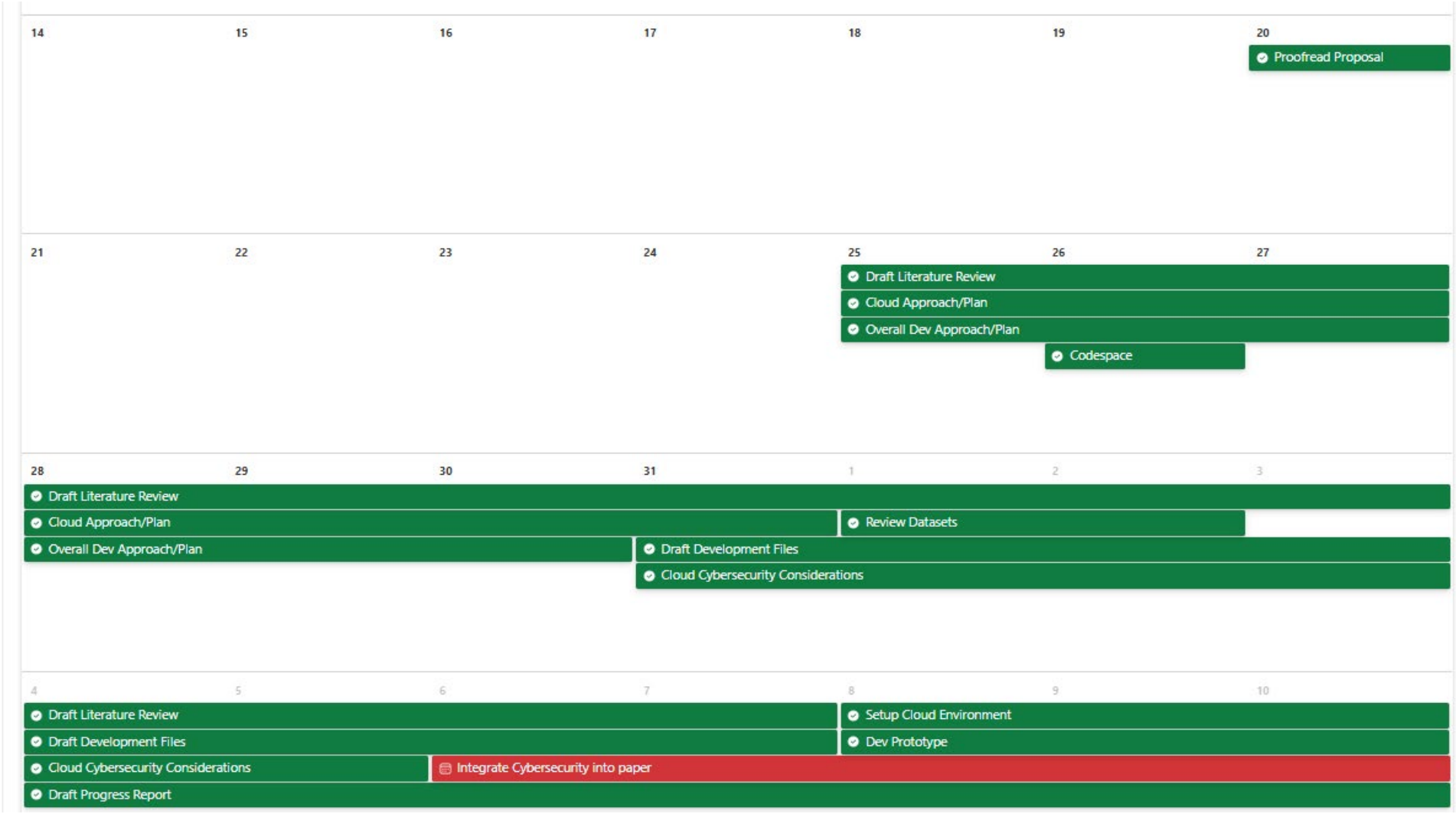


Figure 12 - Prediction Plot: KNN

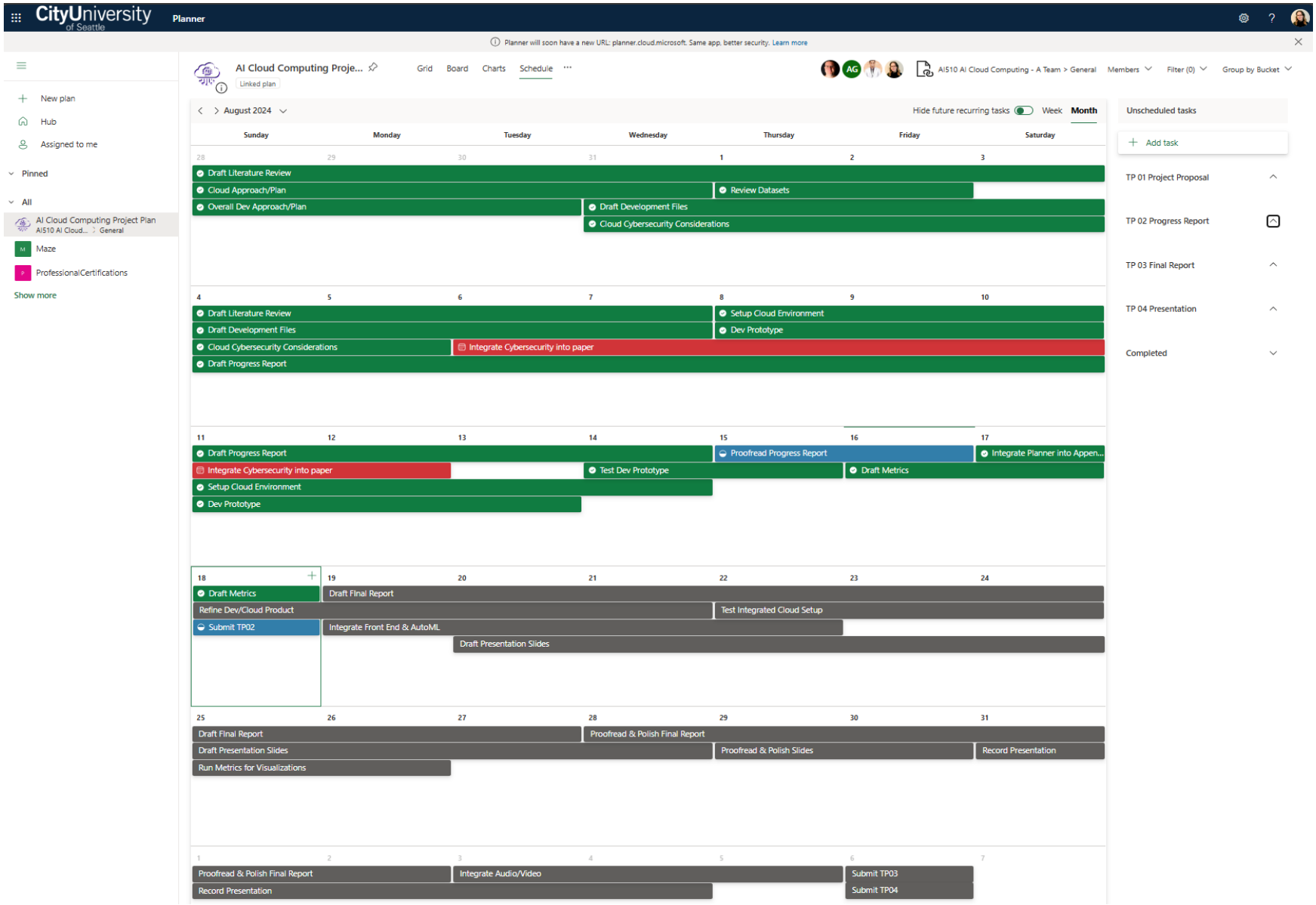
APPENDIX B

Microsoft Planner is tracking the team's plan and progress throughout this team project.

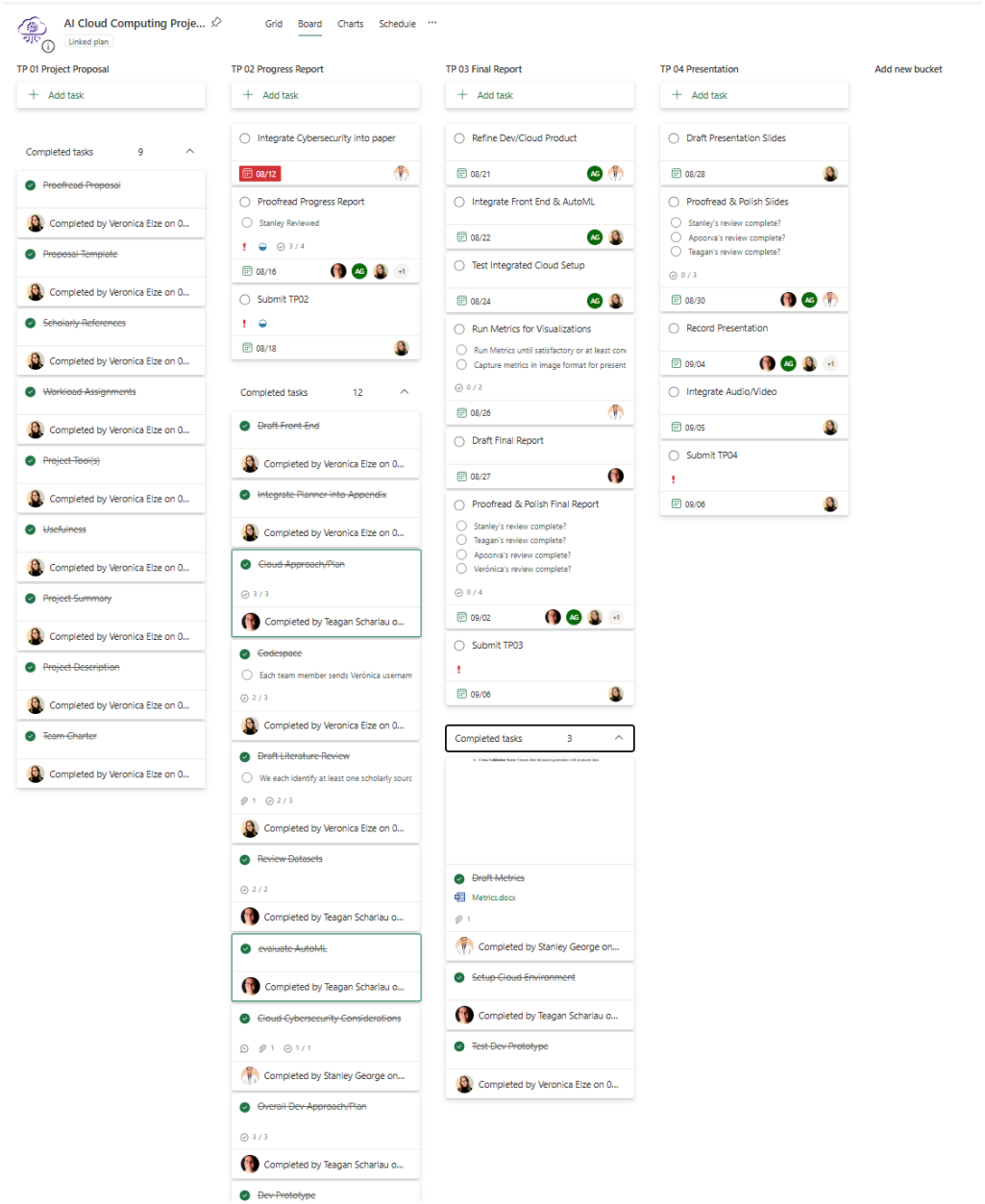
Team Project Plan - July



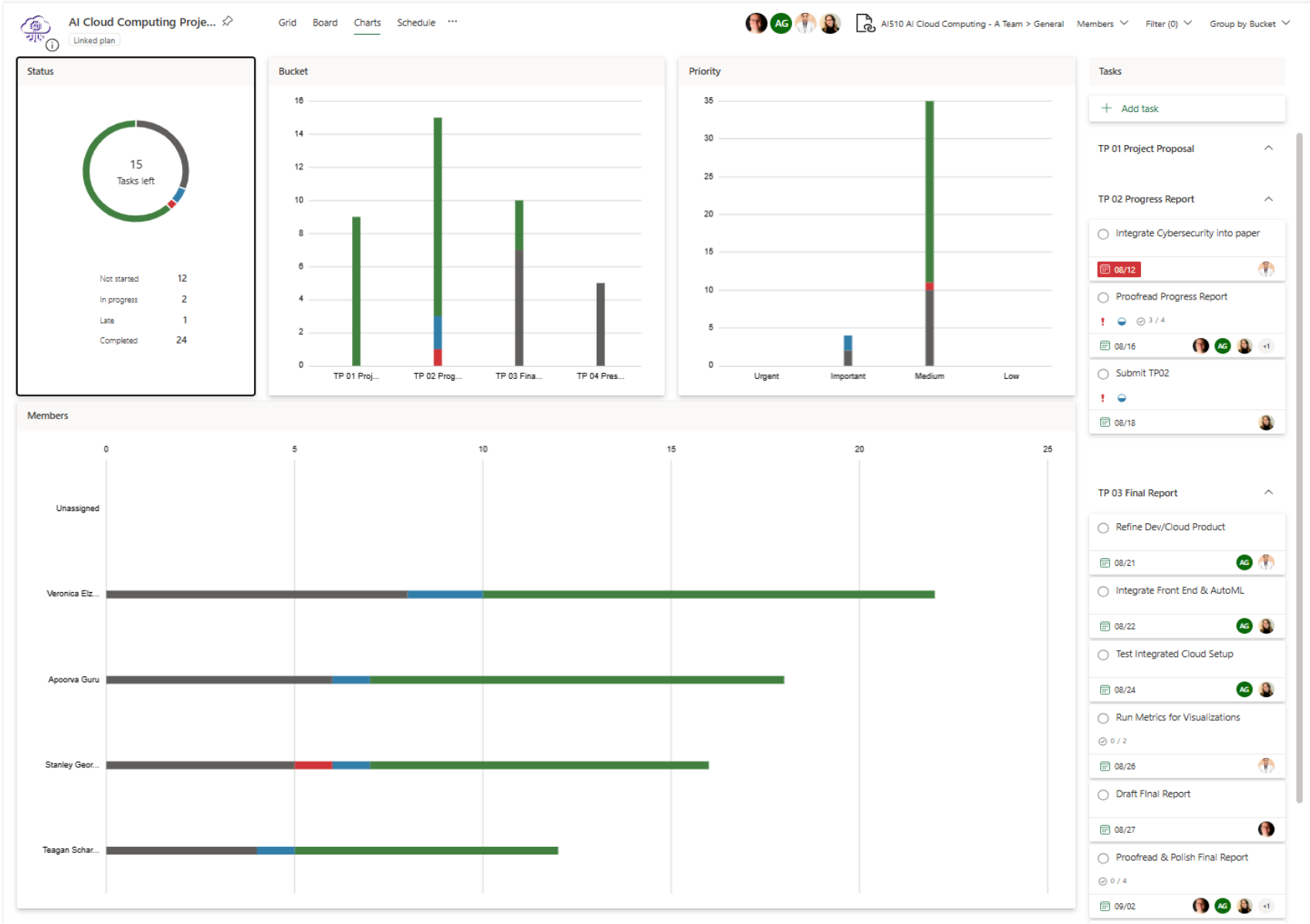
Team Project Plan – August



Team Project Progress – Kanban buckets



Team Project Progress – Charts (quantity, bucket division, priorities, and assignment allocations)



APPENDIX C

All code files currently operate in a local environment. The next phase will involve migrating, improving, and integrating these files into Azure cloud services.

index.html – collect user input, display predicted output

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Predict House Price</title>
    <style>
        /* Basic styling for the body */
        body {
            font-family: Arial, sans-serif;
            background-color: #f4f4f9;
            margin: 0;
            padding: 0;
            display: flex;
            justify-content: center;
            align-items: center;
            height: 100vh;
        }
        /* Styling for the container div */
        .container {
            background-color: #fff;
            padding: 20px;
            border-radius: 8px;
            box-shadow: 0 0 10px rgba(0, 0, 0, 0.1);
            width: 300px;
        }
        /* Styling for the heading */
        h1 {
            text-align: center;
            color: #333;
        }
        /* Styling for the labels */
        label {
            display: block;
            margin-bottom: 5px;
            color: #555;
        }
        /* Styling for the text inputs */
        input[type="text"] {
            width: 95%;
            padding: 8px;
            margin-bottom: 10px;
            border: 1px solid #ccc;
            border-radius: 4px;
        }
        /* Styling for the submit button */
        input[type="submit"] {
            width: 100%;
            padding: 10px;
            background-color: #28a745;
            border: none;
            border-radius: 4px;
            color: #fff;
            font-size: 16px;
            cursor: pointer;
        }
        /* Hover effect for the submit button */
        input[type="submit"]:hover {
            background-color: #218838;
        }
        /* Responsive styling for smaller screens */
        @media (max-width: 600px) {
            .container {
                width: 90%;
                padding: 15px;
            }
            input[type="text"], input[type="submit"] {
                font-size: 14px;
            }
        }
    </style>
</head>
<body>
    <div class="container">
        <h1>Predict House Price</h1>
        <form action="/predict" method="post">
            {% for feature in features %}
            <!-- Loop through features and create a label and input for each -->
            <label for="{{ feature }}">{{ feature_labels[feature] }}:</label>
            <input type="text" id="{{ feature }}" name="{{ feature }}">
            {% endfor %}
            <!-- Submit button for the form -->
            <input type="submit" value="Predict">
        </form>
    </div>
</body>
</html>
```

app.py – incorporates Flask, connects front/back end

```
from flask import Flask, request, render_template
import pandas as pd
import numpy as np
from sklearn.linear_model import LinearRegression

app = Flask(__name__)

# Create feature labels
feature_labels = {
    'bedrooms': 'Number of Bedrooms',
    'bathrooms': 'Number of Bathrooms',
    'sqft_living': 'Living Area (sqft)',
    'sqft_lot': 'Lot Area (sqft)',
    'floors': 'Number of Floors',
    'waterfront': 'Waterfront (Yes=1, No=0)',
    'view': 'View Rating',
    'condition': 'Condition Rating',
    'grade': 'Grade Rating',
    'sqft_above': 'Above Ground Living Area (sqft)',
    'sqft_basement': 'Basement Area (sqft)',
    'yr_built': 'Year Built',
    'yr_renovated': 'Year Renovated',
    'zipcode': 'Zip Code',
    'lat': 'Latitude',
    'long': 'Longitude',
    'sqft_living15': 'Living Area of 15 Nearest Neighbors (sqft)',
    'sqft_lot15': 'Lot Area of 15 Nearest Neighbors (sqft)'
}

# Load the dataset and model
url = "housing.csv"
df = pd.read_csv(url)
df = df.drop(['id', 'date'], axis=1)

# Replace empty strings with NaN and then drop or fill them
df.replace('', np.nan, inplace=True)
df.dropna(inplace=True)

# Calculate the correlation matrix
corr_matrix = df.corr()

# Print the correlation matrix for debugging
print(corr_matrix)

# Adjust the threshold to find highly correlated features
threshold = 0.5
high_corr_features = corr_matrix.index[abs(corr_matrix["price"]) > threshold].tolist()

# Check if 'price' is in the list before removing it
if 'price' in high_corr_features:
    high_corr_features.remove('price')

filtered_feature_labels = {feature: label for feature, label in feature_labels.items() if
feature in high_corr_features}

# Check if high_corr_features is not empty
if not high_corr_features:
    raise ValueError(f"No features found with correlation to 'price' above {threshold}.")

# Prepare the data
X = df[high_corr_features]
y = df['price']

# Check for missing values
if X.isnull().values.any() or y.isnull().values.any():
    raise ValueError("Missing values found in the dataset.")

# Train the model
model = LinearRegression()
model.fit(X, y)

# Dynamically pass feature labels
@app.route('/')
def index():
    return render_template('index.html', features=high_corr_features,
feature_labels=filtered_feature_labels)

# Prediction logic
@app.route('/predict', methods=['POST'])
def predict():
    input_data = [float(request.form[feature]) for feature in high_corr_features]
    input_data = np.array(input_data).reshape(1, -1)
    prediction = model.predict(input_data)[0]
    return f'The predicted house price is: ${prediction:.2f}'

if __name__ == '__main__':
    app.run(debug=True)
```

trainevalmodels.py – loads, trains, and evaluates

```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
import seaborn as sns

# Import necessary machine learning models and metrics
from sklearn.ensemble import RandomForestRegressor  # Add this import
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.neighbors import KNeighborsRegressor

# Load the dataset
url = "Housing.csv"
df = pd.read_csv(url)

# Drop unnecessary columns
df = df.drop(['id', 'date'], axis=1)

# Plot the distribution of housing prices
sns.histplot(df['price'], bins=50, kde=True)
plt.title('Distribution of Housing Prices')
plt.show()

# Plot the correlation matrix
plt.figure(figsize=(14, 10))
corr_matrix = df.corr()
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', fmt='.2f')
plt.title('Correlation Matrix', size=20)
plt.show()

# Select features with high correlation to price
corr_matrix = df.corr()
high_corr_features = corr_matrix.index[abs(corr_matrix['price']) > 0.5]
df_reduced = df[high_corr_features]

# Initialize models
linear_model = LinearRegression()
random_forest_model = RandomForestRegressor(n_estimators=100, random_state=42)
knn_model = KNeighborsRegressor(n_neighbors=5)

# Define features (X) and target (y)
X = df_reduced.drop('price', axis=1)
y = df_reduced['price']

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

# Train the models
linear_model.fit(X_train, y_train)
random_forest_model.fit(X_train, y_train)
knn_model.fit(X_train, y_train)

# Make predictions
linear_preds = linear_model.predict(X_test)
rf_preds = random_forest_model.predict(X_test)
knn_preds = knn_model.predict(X_test)

# Calculate performance metrics
linear_rmse = np.sqrt(mean_squared_error(y_test, linear_preds))
linear_r2 = r2_score(y_test, linear_preds)

rf_rmse = np.sqrt(mean_squared_error(y_test, rf_preds))
rf_r2 = r2_score(y_test, rf_preds)

knn_rmse = np.sqrt(mean_squared_error(y_test, knn_preds))
knn_r2 = r2_score(y_test, knn_preds)

# Print performance metrics
print(f'Linear Regression RMSE: {linear_rmse}, R^2: {linear_r2}')
print(f'Random Forest RMSE: {rf_rmse}, R^2: {rf_r2}')
print(f'KNN RMSE: {knn_rmse}, R^2: {knn_r2}')

# Plot predictions vs actual values
plt.figure(figsize=(21, 7))

plt.subplot(1, 3, 1)
plt.scatter(y_test, linear_preds, alpha=0.3)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'k--', lw=2)
plt.xlabel('Actual')
plt.ylabel('Predicted')
plt.title('Linear Regression Predictions')

plt.subplot(1, 3, 2)
plt.scatter(y_test, rf_preds, alpha=0.3)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'k--', lw=2)
plt.xlabel('Actual')
plt.ylabel('Predicted')
plt.title('Random Forest Predictions')

plt.subplot(1, 3, 3)
plt.scatter(y_test, knn_preds, alpha=0.3)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'k--', lw=2)
plt.xlabel('Actual')
plt.ylabel('Predicted')
plt.title('KNN Predictions')

plt.tight_layout()
plt.show()
```